



Traduction des expressions Bloc d'activation Entrées-sorties

Sommaire

Traduction des expressions

[Bloc d'activation](#)

[Entrées-sorties](#)

Calcul arithmétique en **nasm**

```
mov rax, qword [a]
imul rax, qword [d]
mov rcx, qword [b]
imul rcx, qword [c]
sub rax, rcx
```

Calculer $ad-bc$

Problèmes de traduction

Le choix du registre pour calculer ad n'est pas indépendant du choix du registre pour bc

Certaines instructions contiennent à la fois un opérateur et un accès mémoire à un opérande

Calcul par la pile

```
mov rax, qword [a]
push rax
mov rax, qword [d]
push rax
```

```
pop rcx
pop rax
imul rax, rcx
push rax
```

```
mov rax, qword [b]
push rax
mov rax, qword [c]
push rax
```

```
pop rcx
pop rax
imul rax, rcx
push rax
```

```
pop rcx
pop rax
sub rax, rcx
push rax
```

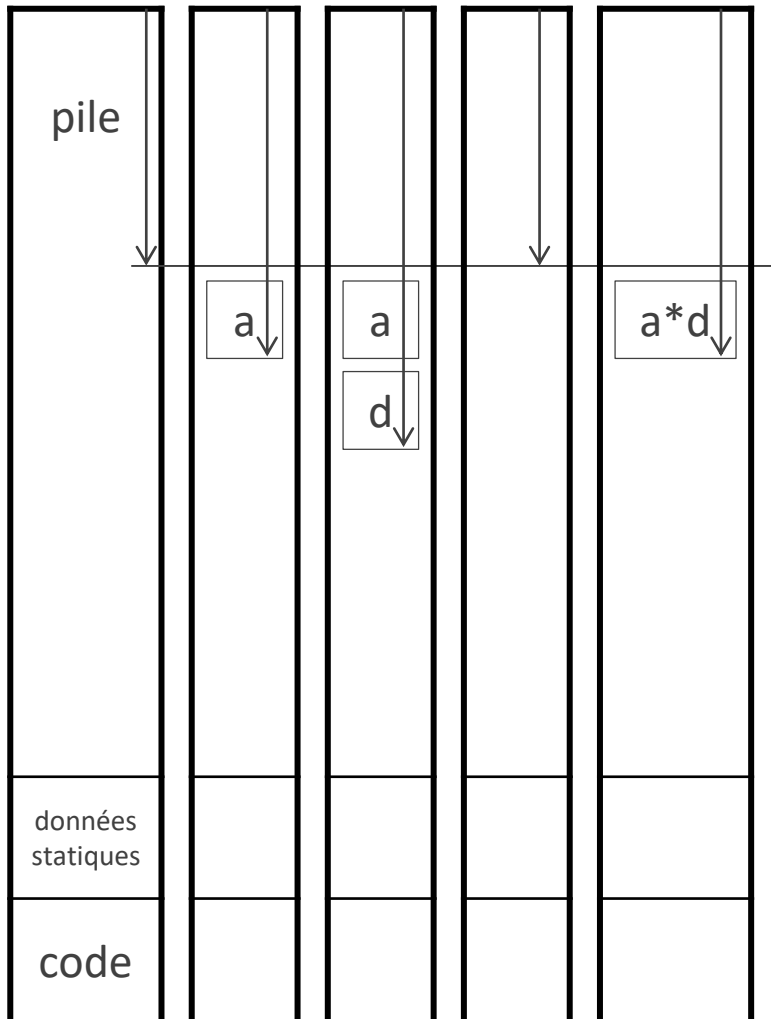
Une méthode de calcul qui facilite la traduction

Pour chaque opération arithmétique, le choix du registre est indépendant des autres opérations

Les instructions qui contiennent un opérateur sont séparées des instructions qui vont chercher un opérande

Exemple

Calculer $ad-bc$



Traduire des expressions en code qui évalue l'expression en utilisant la pile

Étapes

Évaluer tous les opérandes

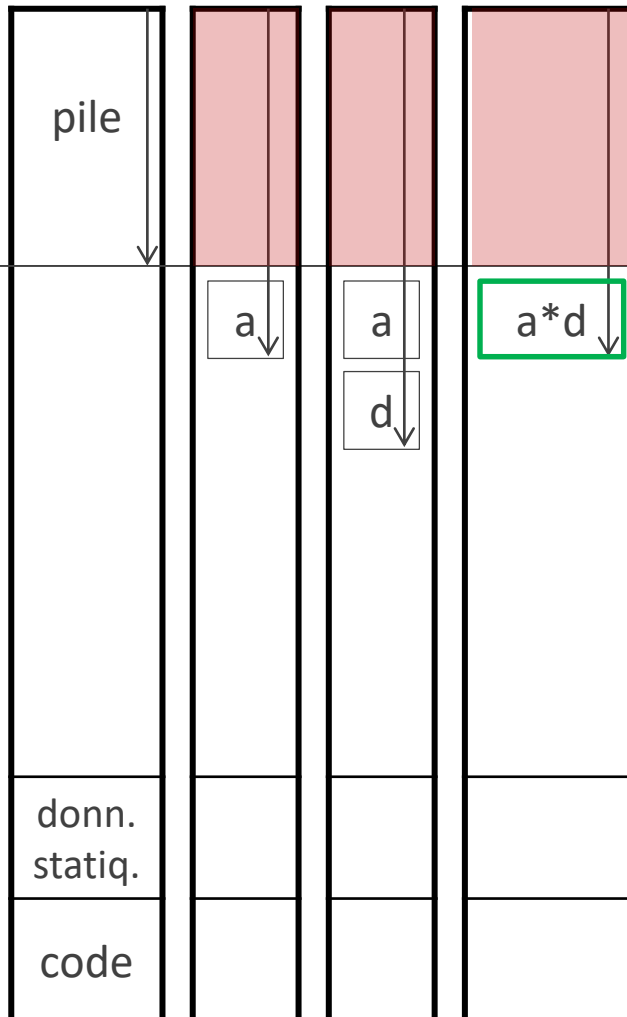
Dépiler les opérandes dans des registres différents

Appliquer l'opérateur

Empiler le résultat

Traduire des expressions en code qui évalue l'expression en utilisant la pile

$sp0$, position du sommet de pile au début de l'évaluation



Pourquoi ça marche récursivement

Le code qui évalue une expression ne change pas ce qui était déjà dans la pile au moment où l'évaluation a commencé

→ Le calcul de d n'écrase pas a

À la fin, la valeur de l'expression est dans la pile juste au-dessus du niveau $sp0$

→ On sait où sont a et d quand on en a besoin

Étapes

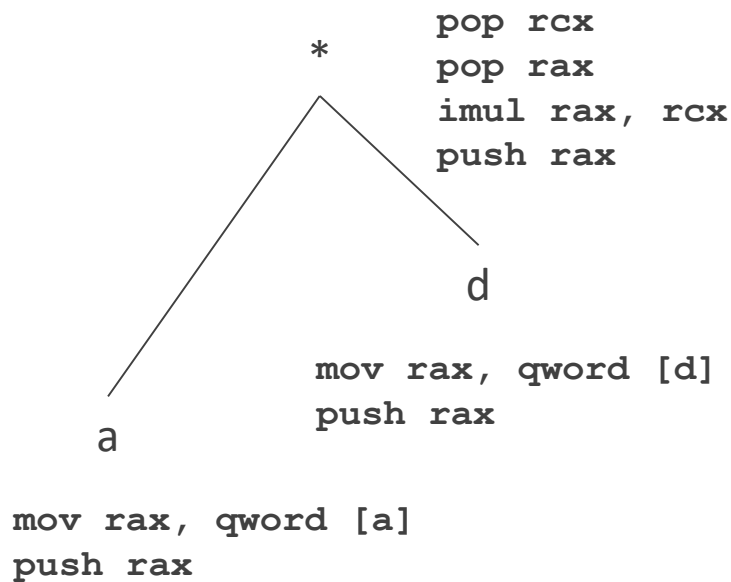
Évaluer tous les opérandes

Dépiler les opérandes dans des registres différents

Appliquer l'opérateur

Empiler le résultat

Traduction des expressions



Pour un nœud contenant un opérateur

Traduction des opérandes

Dépiler les valeurs des opérandes dans le bon ordre

Appliquer l'opérateur

Empiler le résultat

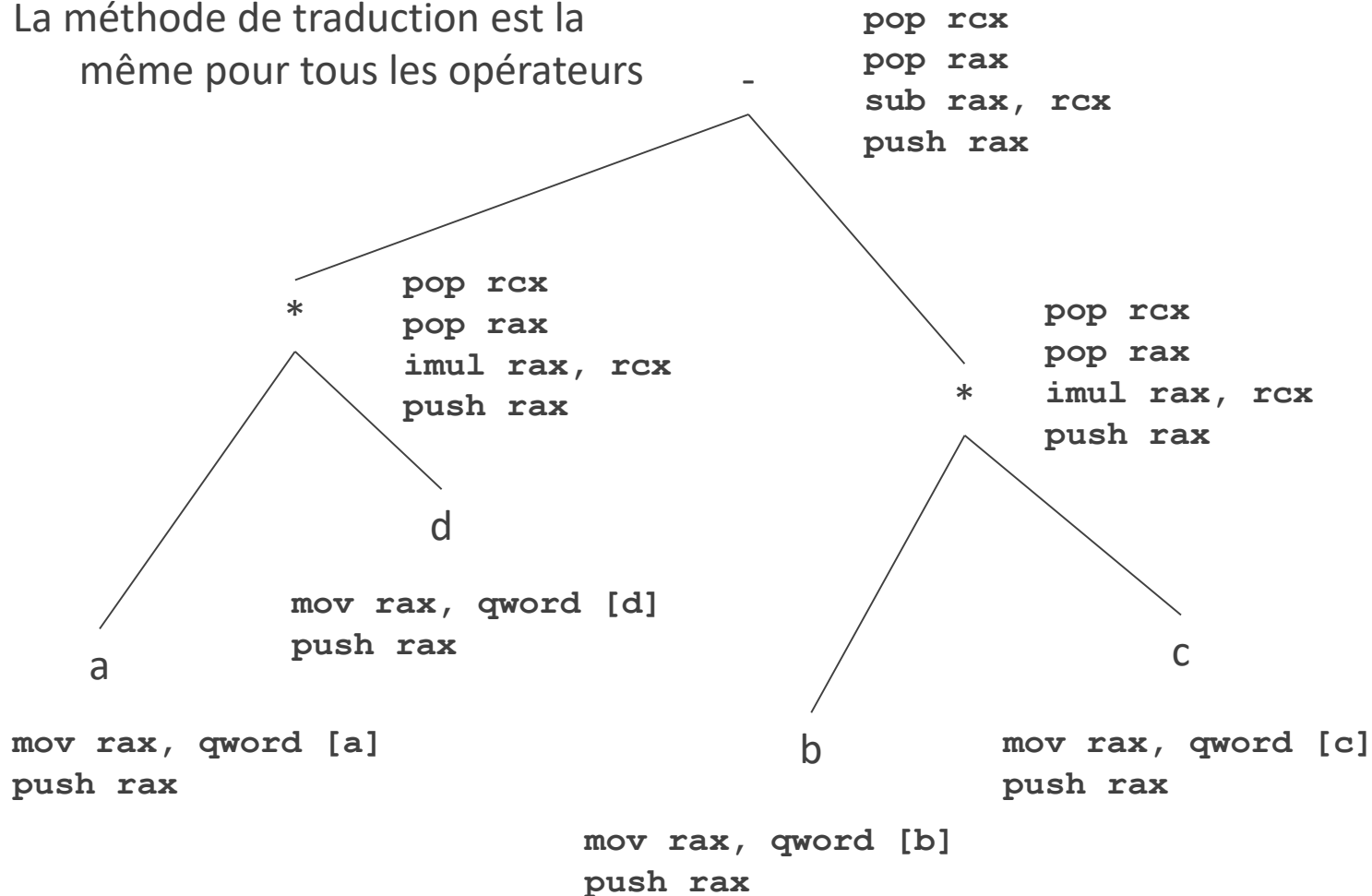
Pour un nœud feuille

Pour un identificateur, charger la valeur dans un registre

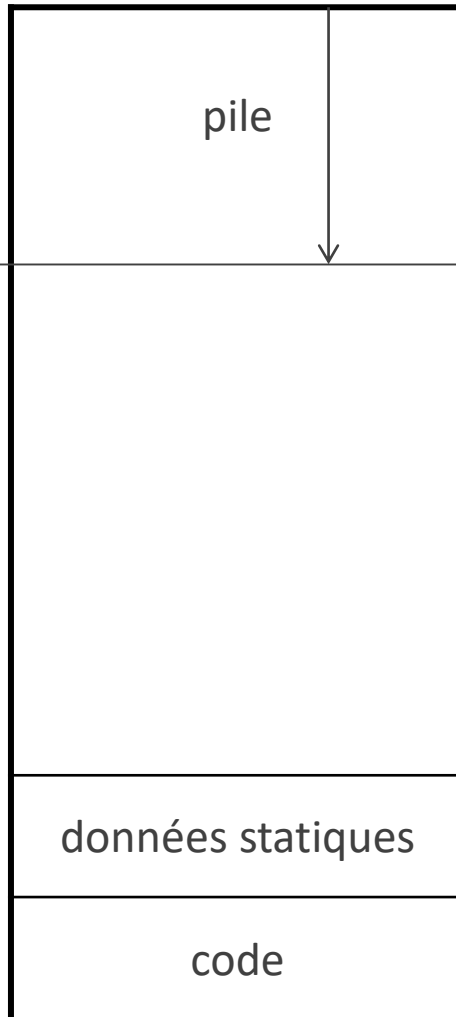
Empiler la valeur

Traduction des expressions

La méthode de traduction est la même pour tous les opérateurs



sp0, position
du sommet
de pile au
début de
l'évaluation



Traduire des expressions en code qui évalue l'expression en utilisant la pile

Traduire du C en code nasm 64 bits

1. `a`
2. `5`
3. `b*c*(b+c);`
4. `a=b-2;`
5. `a=(b=c);`

Sommaire

Traduction des expressions

Bloc d'activation

Entrées-sorties

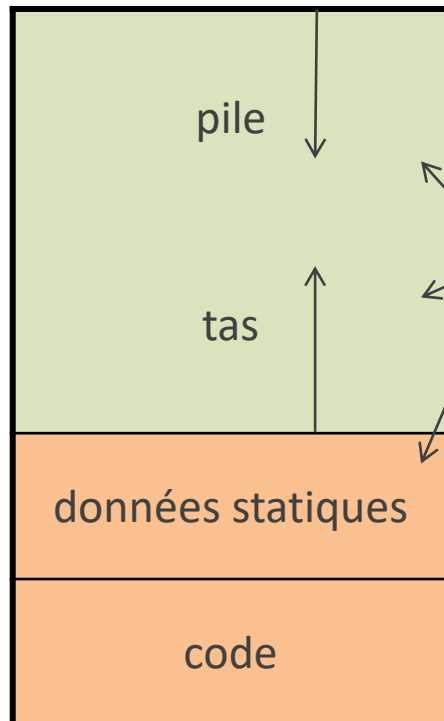
Allocation automatique

adresses

hautes



basses



Allocation statique

Allocation dynamique

Sur demande explicite dans le source

Allocation automatique : dans la pile

Il y a plus d'espace disponible que dans les registres

Les variables peuvent être locales à une fonction :
quand la fonction se termine, la mémoire est libérée

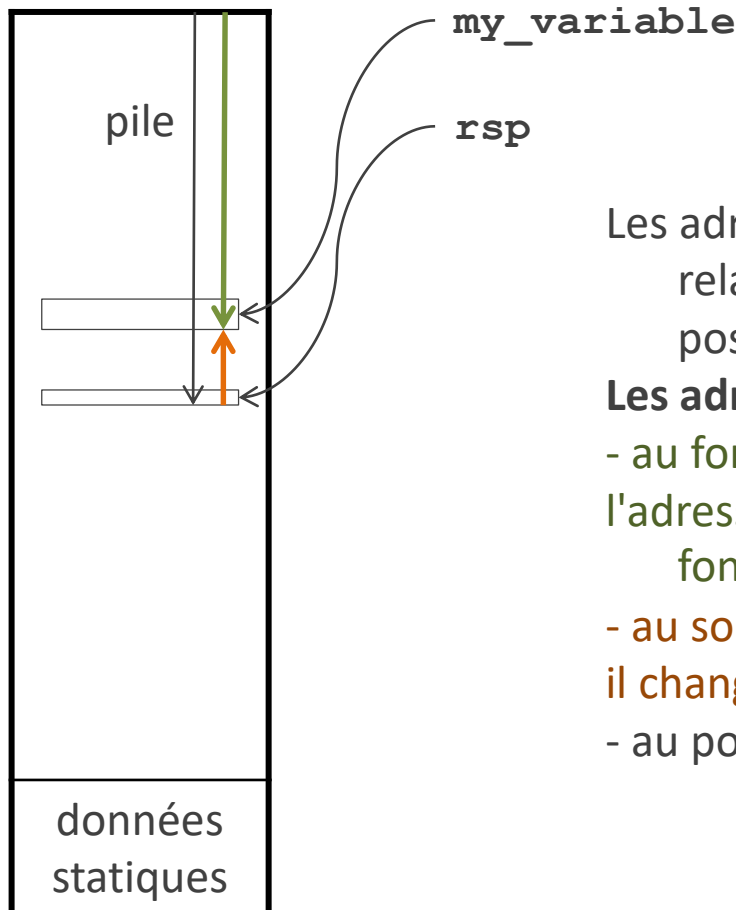
Compatible avec les fonctions récursives

Allocation automatique

adresses

hautes

basses



Les adresses dans la pile sont des adresses relatives (*offsets*) par rapport à une certaine position dans la pile

Les adresses sont relatives à quoi ?

- au fond de la pile ?

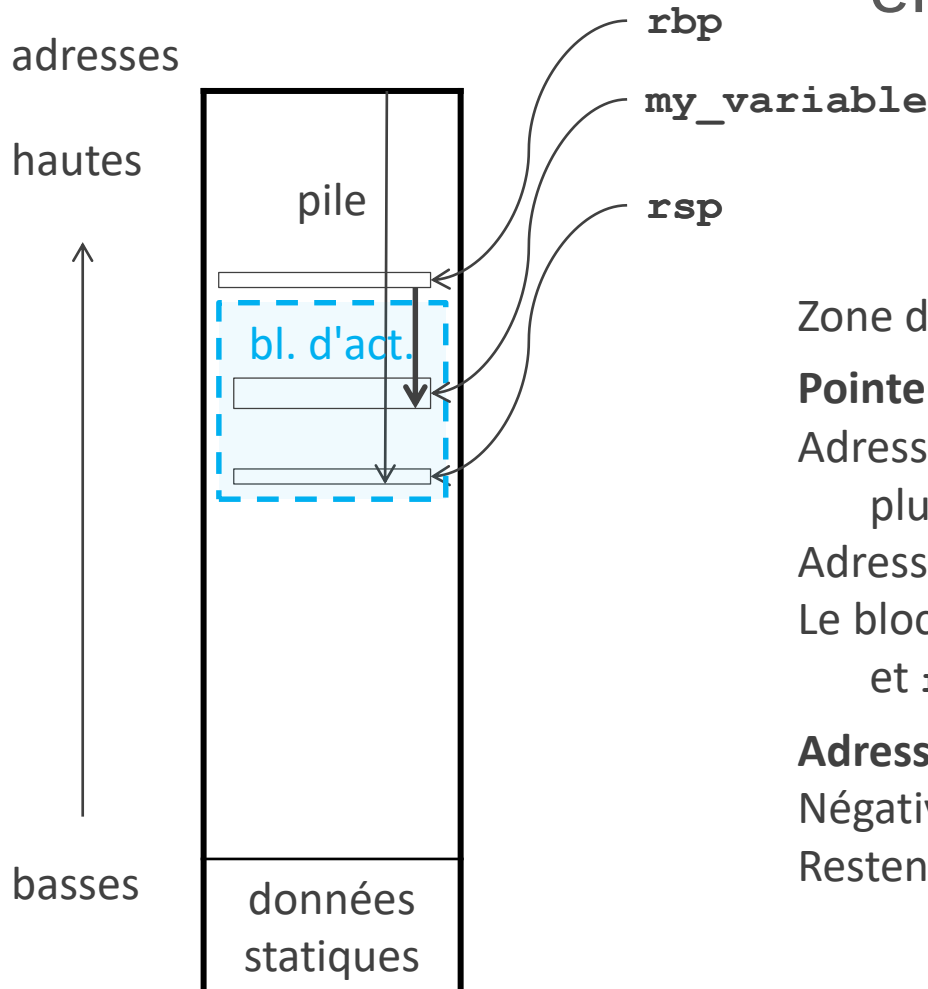
l'adresse relative changerait avec chaque appel de fonction

- au sommet de pile ?

il change tout le temps avec les **push** et les **pop**

- au pointeur de base **rbp**

Bloc d'activation ou enregistrement d'activation (*stack frame*)



Zone de mémoire sous le sommet de pile

Pointeur de base

Adresse de la base du bloc d'activation : un point plus stable que le sommet de pile

Adresse contenue dans le registre `rbp`

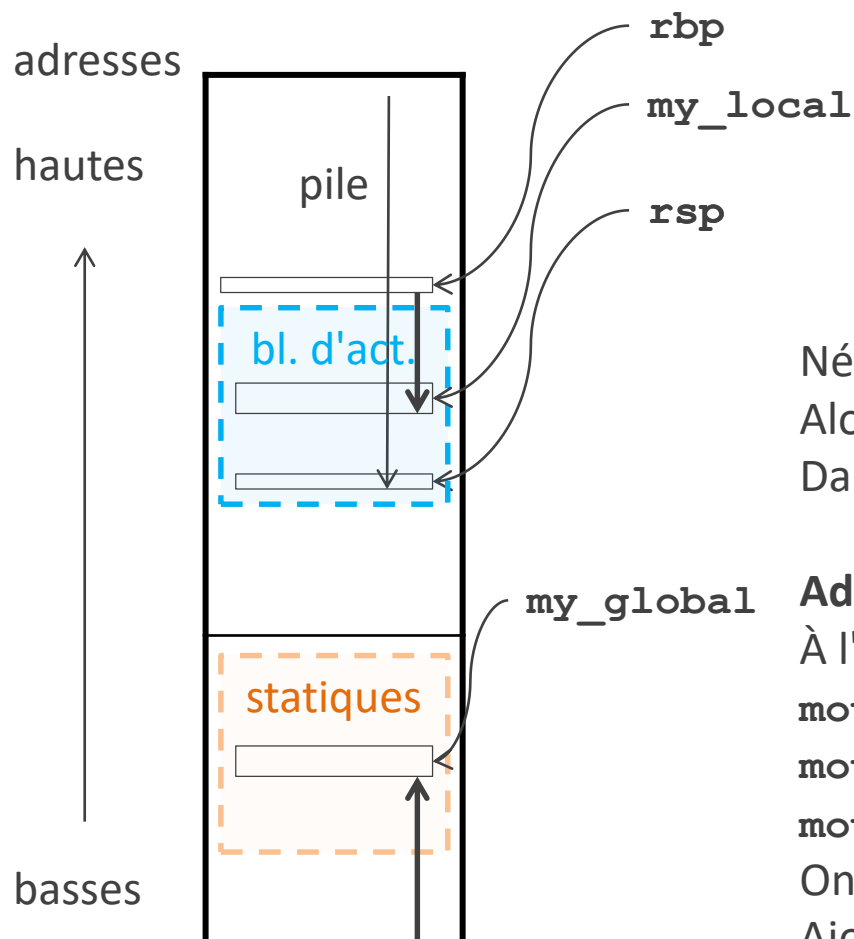
Le bloc est entre les adresses `rbp` (non comprise) et `rsp` (comprise)

Adresses relatives dans le bloc d'activation

Négatives par rapport à `rbp`

Restent valides malgré les `push` et les `pop`

Adresses relatives des données locales



Négatives

Alors que les adresses statiques sont positives
Dans les deux cas, l'adresse est celle du premier octet

Adresses effectives

À l'intérieur des crochets

`mov qword [rbp-24], 0` ✓

`mov qword [rbp+rax], 0` ✓

`mov qword [rbp-rax], 0` ✗

On peut soustraire une constante, pas un registre
Ajouter un registre à valeur négative

Utiliser le bloc d'activation

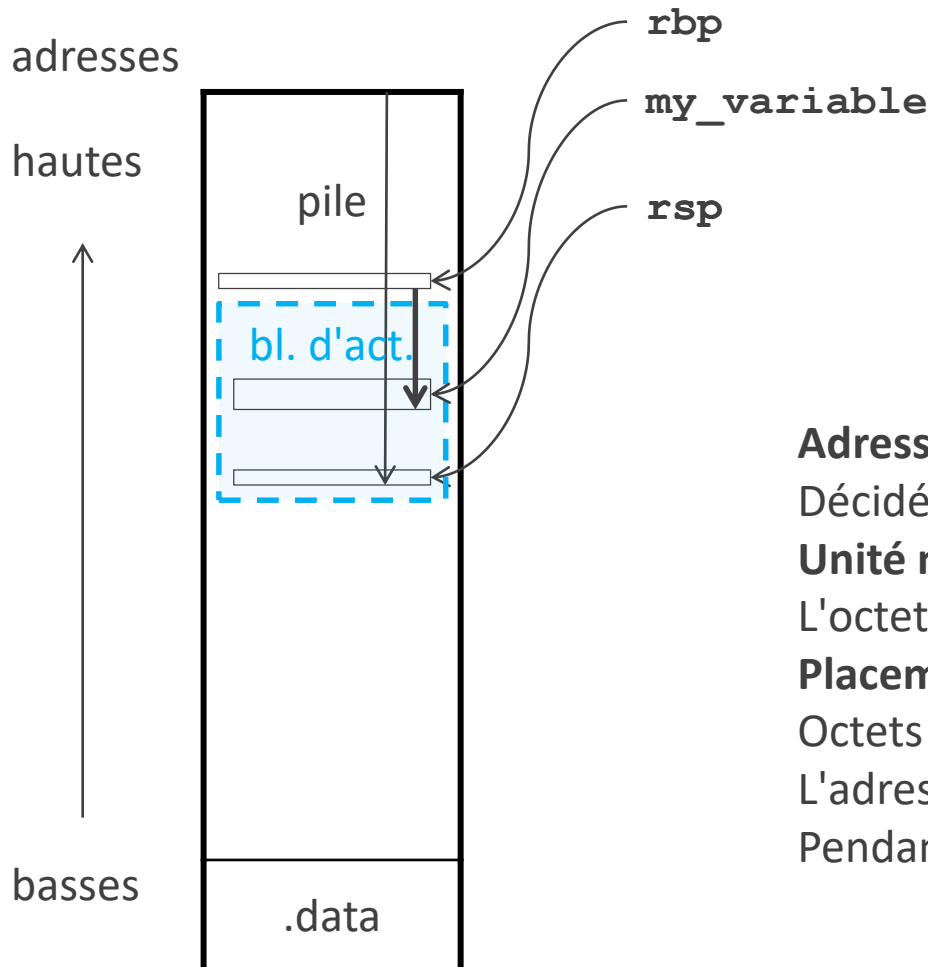
```
push rbp
mov rbp, rsp
sub rsp, 8
mov qword [rbp-8], 0 ; sum = 0
loop_condition:
    cmp rdi, 0
    jle end_loop
add qword[rbp-8], rdi ; sum += rdi
dec rdi
jmp loop_condition
end_loop:
mov rax, qword[rbp-8] ; rax = sum
mov rsp, rbp
pop rbp
```

On utilise l'adresse par rapport à **rbp**
et non par rapport à **rsp**

Pendant la durée de vie d'un bloc
d'activation,

- **rbp** ne change jamais
- **rsp** change dès qu'on utilise la pile pour faire un calcul ou sauvegarder une donnée

Disposition des données locales



Adresses relatives

Décidées à la compilation

Unité minimale d'adressage

L'octet (*byte*)

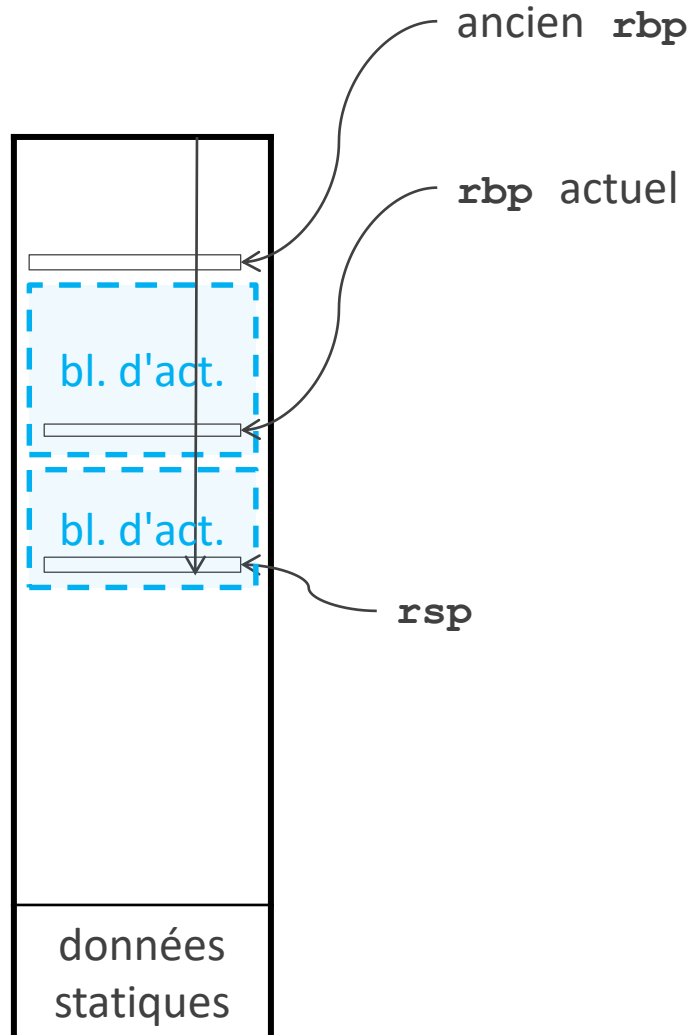
Placement d'un objet

Octets consécutifs (la taille dépend du type)

L'adresse est celle du premier octet

Pendant l'analyse des déclarations

Pile de blocs d'activation



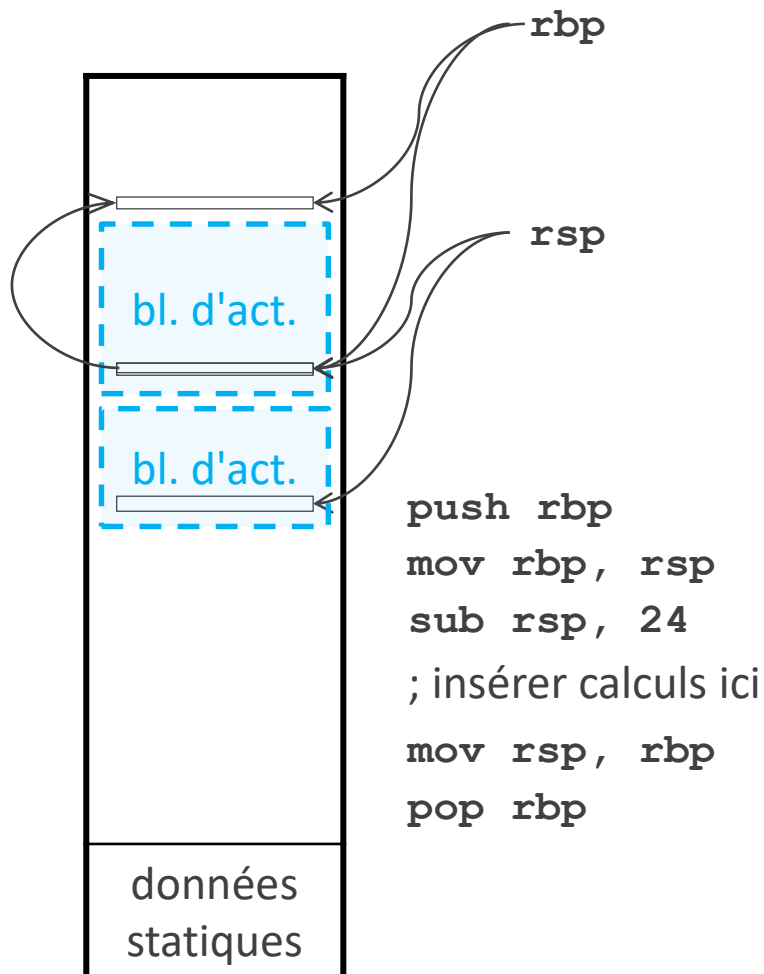
Réserver un nouveau bloc d'activation

On n'a plus directement accès aux variables du bloc précédent

Libérer le bloc d'activation courant

Restaurer l'ancienne valeur de `rbp`

Pile de blocs d'activation



Réserver un bloc d'activation

Sauvegarder le `rbp` précédent sur la pile (lien de contrôle, *control link*)

Écraser `rbp` avec sa nouvelle valeur

Déplacer `rsp` de la taille du nouveau bloc

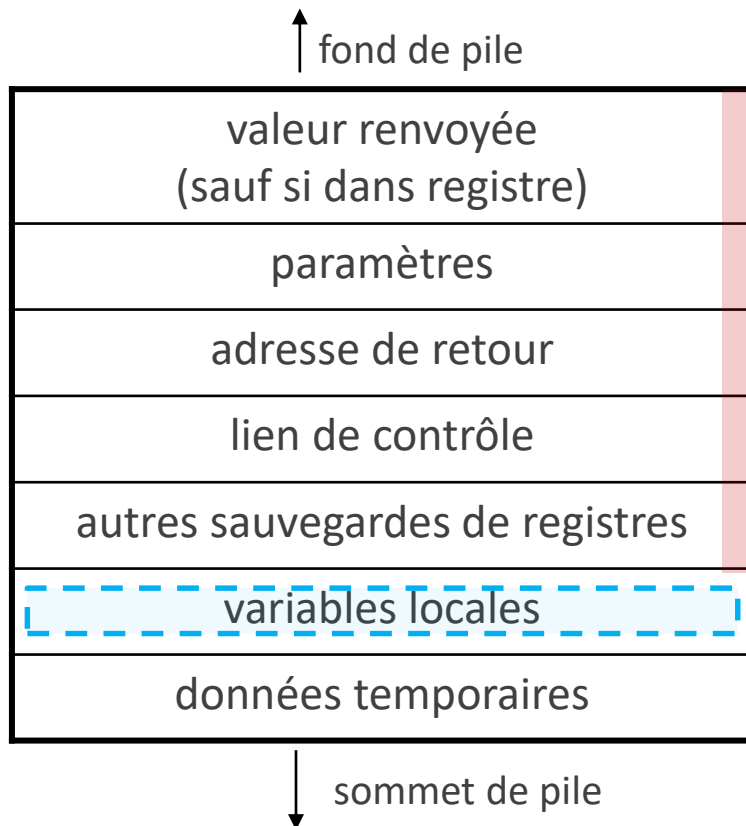
Libérer un bloc d'activation

Restaurer le `rsp` précédent

Restaurer le `rbp` précédent

Bloc d'activation ou enregistrement d'activation (*stack frame*)

Zone de mémoire empilée à l'appel d'une fonction et dépilée au retour



Lien de contrôle : pointe sur la **base** du bloc d'activation appelant

Sauvegarde des **registres non volatils** au moment de l'appel

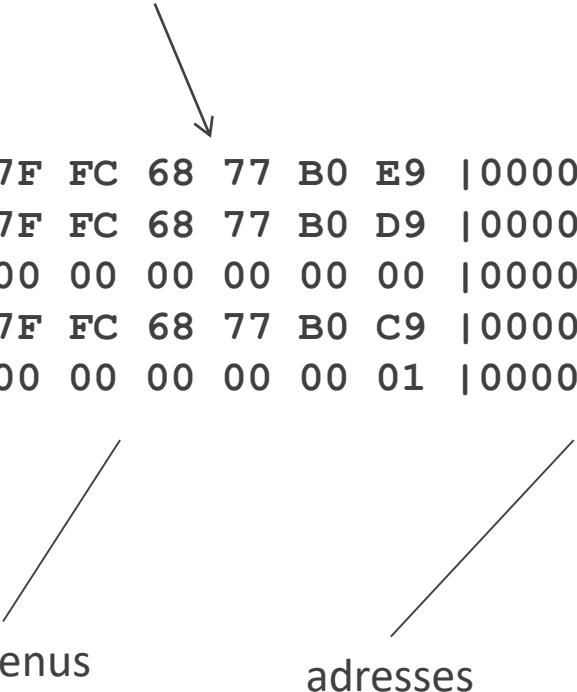
Données temporaires : pour le calcul des expressions

Pourquoi la valeur de retour au-dessous de tout le reste ?

Pourquoi les paramètres au-dessous de l'adresse de retour ?

```
mov rdi, 5
call stackview
```

Aide sur la pile



00	00	7F	FC	68	77	B0	E9		00007FFC68778FE0
00	00	7F	FC	68	77	B0	D9		00007FFC68778FD8
00	00	00	00	00	00	00	00		00007FFC68778FD0
00	00	7F	FC	68	77	B0	C9		00007FFC68778FC8
00	00	00	00	00	00	00	01		00007FFC68778FC0

contenus

adresses

Bibliothèque de fonctions en nasm : cpu2.0
<https://sourceforge.net/projects/baselibs/>
 Visualiser la pile : stackview

Sommaire

Traduction des expressions

Bloc d'activation

Entrées-sorties

Entrées-sorties avec `syscall`

Appel système	<code>rax</code>	<code>rdi</code>	<code>rsi</code>	<code>rdx</code>
<code>exit</code>	60	valeur de retour		
<code>read</code>	0	fichier : 0	adr. destination	taille
<code>write</code>	1	fichier : 1	adr. source	taille

On met plusieurs informations dans des registres avant d'invoquer `syscall`

La valeur dans `rax` détermine quel appel système est réalisé : `exit`, `read`, `write`...

Si `read` ou `write` :

- dans quel fichier ? `stdin: 0`
`stdout: 1`
- à quelle adresse ?
- taille des données ?

```

mov rax, 1
mov rdi, 1
mov rsi, my_string
mov rdx, 5
syscall

```

Entrées-sorties par caractères

```
push 'v'  
mov rax, 1  
mov rdi, 1  
mov rsi, rsp  
mov rdx, 1  
syscall
```

```
sub rsp, 8  
mov rax, 0  
mov rdi, 0  
mov rsi, rsp  
mov rdx, 1  
syscall  
pop rax
```

Lire ou afficher des caractères

Instruction `syscall` avec 0 (lire) ou 1 (écrire)
dans `rax` et `rdi`

Afficher un entier en décimal

initialiser i à imax

faire :

diviser [number] par 10

copier le quotient dans [number]

ajouter 48 au reste

copier le résultat dans [digits+i]

décrémenter i

tant que [number] est > 0

écrire [digits+i+1] ... [digits+imax]

écrire 10

Lire au clavier un entier en décimal

initialiser [number] à 0

lire un caractère dans [digit]

tant que [digit] est un chiffre :

soustraire 48 de [digit]

multiplier [number] par 10

ajouter [digit] à [number]

Entrées-sorties : entiers en décimal

Décomposer chiffre par chiffre

Gérer + et -

Attention, `syscall` écrase `rcx` et `r11`

Merci

CONTACT

ERIC LAPORTE

00 +33 (0)1 60 95 75 52

ERIC.LAPORTE@UNIV-EIFFEL.FR